P1053R1
Future-proofing continuations for executors

Published Proposal, 24 June 2018

This version:
https://wg21.link/P1053R1

Authors:
Lee Howes (Facebook)
Eric Niebler (Facebook)

Audience:
SG1, LEWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
https://github.com/LeeHowes/CPP/blob/master/future_continuations.bs

---

Table of Contents

Contributors:

- Jay Feldblum
- Andrii Grynenko
- Kirk Shoop

§ 1. Changelog

§ 1.1. Revision 1

- Added a before/after comparison.
- Added TLDR to introduction.
- Added concrete bulk design.
- Proposed a more flexible bulk interface.
- then_value -> on_value for consistency with p1054.

§ 1.2. Revision 0

- Initial design

§ 2. Introduction and TLDR

p0443 defines interfaces for executors and the continuation functions passed to them. p1054 utilises these fundamental interfaces to build expressive concepts for future types where continuations are cleanly mapped through continuation construction functions.

The current design of the continuation functions passed to then_execute are based on the ability of the executor to invoke the continuation.

In essence the continuations have an interface similar to:

```
struct callable {
  R operator()(T);
  R operator()(exception_arg, e);
};
```

where either function is optional, and in that situation the other operation will act as a passthrough. One reason for designing the API in this way is to allow a simple lambda function to be passed to `then_execute`:

```
e.then_execute([](T value){return value;}, input_future);
```

The downsides of this design are twofold:

- The description of the continuation is based on ability to invoke it. There is then potential for errors that would easily slip through code review, and silently cause unexpected runtime behaviour.
- The mechanism of describing the continuation with two parallel end-to-end data paths removes the ability to catch and pass an exception from the value operator, or to log and passthrough an exception from the exception operator without rethrowing the exception.

On the first point, consider the following struct that an author might write in an attempt to handle both values and exceptions at some stage in the pipeline:

```
struct callable {
  int operator()(int value) {
    return value + 1;
  }
  int operator()(std::exception_ptr e) {
    return 0;
  }
};
```

This is a trivial example of ignoring the precise exception and attempting to recover. Note that the reality here, based on the p0443 definition is that the exception function is not callable as the `EXCEPTIONAL` case. It will therefore not be called and an exception will bypass. In effect, this struct is semantically equivalent to:

```
struct callable {
  int operator()(int value) {
    return value + 1;
  }
  int operator()(exception_arg, std::exception_ptr e) {
    std::rethrow_exception(e);
  }
};
```

where we have silently lost our recovery path and passed the error through with potentially negative consequences. There is no compilation or runtime error here, and this kind of problem could be hard to catch in code review.

On the second point, consider an exception handler that only exists to log that an exception reached a point in the stream:

```
struct callable {
  int operator()(int value) {
    return value + 1;
  }
  int operator()(exception_arg, std::exception_ptr e) {
    std::cerr << "Have exception\n";
    std::rethrow_exception(e);
  }
};
```

This is an expensive means of doing nothing to the exception. With potential extensions to `std::exception_ptr` that would allow peeking at the exception without rethrow, for example p1066, there is potentially a wide range of optimisations that we lose the ability to perform.

What we might prefer, would be to implement this as:

```
struct callable {
  int operator()(int value) {
    return value + 1;
  }
  std::exception_ptr operator()(exception_arg, std::exception_ptr e) {
    std::cerr << "Have exception\n";
    return e;
  }
};
```

but then we lose the ability to recreate the value.

We consider these two flaws, one of safety and the other of flexibility, as unfortunate limitations of a low-level API like executors.

Expected use of `FutureContinuation` as discussed in p1054 is through the use of helper functions such as `on_value` and `on_error` that take a constrained callable and return a `FutureContinuation`. With these helper functions, and the clean readable code they lead to, there is no need to simplify the `FutureContinuation` to be a trivial callable, and we gain a lot of flexibility by consciously deciding to not simplify it in that way.

We propose to solidify the `FutureContinuation` definition into a more fundamental and general task interface, without overload-resolution complexities. Rather than passing simple callables into the executor and future APIs, we propose a concrete continuation concept, based on promises. The API of a continuation, from the point of view of the input data, is:

```
void set_value();
template<class T>
void set_value(T&&);
void set_exception(std::exception_ptr&&);
```

Bulk is also supported, but orthogonally to that fundamental data transfer API, which allows bulk tasks to be passed through higher-level dataflow-oriented abstractions such as futures..

Syntactically, this proposal suggests only minor changes, and so usability for any direct user of executors is minimally affected:

Comment	Before	After

One way	`executor.execute([](){task();});`	`executor.execute(`
		` on_value([](){`
		`  task();`
		` }));`

Bulk one way	`executor.bulk_execute(`
	` [&out](`
	`  int idx,`
	`  long& shr){`
	`  out+=idx * shr;`
	` },`
	` int{20},`
	` []() -> long {return 3;});`
		`executor.execute(`
		` bulk_on_value(`
		`  [&out](`
		`  int idx,`
		`  long& shr){`
		`  out+=idx * shr;`
		` },`
		` int{20}, // Shape`
		` []() -> long {return 3;}));`

Two way	`auto f = executor.twoway_execute(`
	` [](){return task();});`
		`auto f = executor.twoway_execute(`
		` on_value([](){`
		`  return task();`
		` }));`

Bulk two way	`auto f = executor.bulk_twoway_execute([](int idx, int& out, long& shr){ out+=idx*shr; }, int{20}, [] () -> int {return 0;}, [] () -> long {return 3;});`	`auto f = executor.twoway_execute( bulk_on_value( [](int idx, int &out, long& shr){ out+=idx*shr; }, int{20}, // Shape [] () -> int {return 0;}, [] () -> long {return 3;}));`
Then	`auto f = executor.then_execute( [](T t){return t;}, std::move(fut));`	`auto f = executor.then_execute( on_value([](auto t){ return t; }), std::move(fut));`
Bulk	`auto f = executor.bulk_then_execute([]( const int& in, int idx, int& out, long& shr){ out+=in*(idx+shr); }, int{20}, std::move(fut), [] () -> int {return 0;}, [] () -> long {return 3;});`	`auto f = executor.then_execute( bulk_on_value( []( const int& in, int /*idx*/, int &out, long& shr){ out+=in*(idx+shr); }, int{20}, // Shape [] () -> int {return 0;}, [] () -> long {return 0;}), std::move(fut));`

Future's continuation methods would be similarly modified to take the same continuation concepts, which pass directly down to executors. For example:

Before	After
`auto f = makeFuture<int>(3) .via(e) .then([](int a){return a;});`	`auto f = makeFuture<int>(3) | via(e) | on_value([](int a){return a;});`

Note that futures do not have to be expanded to support bulk explicitly, nor for any similar future concepts we add beyond bulk. This should lead to simpler expansions of the design in the future.

The same modifications we propose in this paper apply to both p0443 and p1054. Uses of the types in p1054 are unaffected but the description of the calling mechanism, return values of the construction functions (`on_value`, `on_error`) and precise semantics would require updates similar to those we propose for p0443.

The rest of this paper aims to justify the above design by explaining how we derive it, followed by concrete proposed wording.

3. Requirements

If we look at some example continuation constructions based on those in p1054 we can see what kind of functionality we might want here.

3.1. on_value

This is the simple passthrough of the exception, while applying some operation to the value.

The callback we expect to create looks like:

```
on_value([](T value){operation(value);});
```

As a flow diagram, something like:

```
set_value --------- Perform Operation ----- return_value

set_exception ------------------------------ return_exception
```

3.2. on_error

The equivalent where we apply some operation to the exception, but not the value. A good example of this might be error recovery. Note that in this case we are breaking the exception chain. The callback we expect to create looks like:

```
on_error([](std::exception_ptr e){
  try {
   std::rethrow_exception(e);
  } catch(recoverable_exception) {
    return 0;
  } catch(...) {
    std::rethrow_exception(std::current_exception());
  }});
```

Or:

```
set_value ------------------------------------------------------/----- return_value
                                                 /----- Recover -----/
set_exception ----- Perform Operation ----- |
                                                 \----- Do not recover ----- return_excep
```

Note that in this case we rethrow twice. Logically the first is just to check the exception type. The second is just returning the exception and relying on external logic to catch as we do not have two outputs in the syntax. Improvements to `exception_ptr` (along the lines of folly's exception_wrapper or those proposed in p1066) could mitigate the first. Ability to return either an `exception_ptr` or a `T` from the error case could remove the second throw.

§ 3.3. on_variant

Here our operation might take a variant of a value and an exception so that we can write a single function that decides what to do: The callback we expect to create looks like:

```
on_variant([](std::variant<T, std::exception_ptr> v){operation(v);});
```

Diagrammatically:

```
set_value -----\                              /----- return_value
               |----- Perform Operation -----|
set_exception -/                              \----- return_exception
```

This is a very common pattern in Facebook's code where `folly::Try<T>`, which carries a value and exception, is the preferred means of parameterising future continuations.

§ 3.4. on_value_with_exception_log

Here we merely log the existence of an error, and pass it through. We might write this as:

```
on_value_with_exception_log(
  [](T value){operation(v);},
  [](std::exception_ptr e){std::cerr << "Have an exception\n"; return e;});
```

Here we have a very simple pair of parallel operations:

```
set_value --------- Perform Operation ----- return_value

set_exception ----- Log ------------------- return_exception
```

Note though that it relies on allowing return of an `exception_ptr` from the exception path to do this without a throw.

§ 4. Concept

As an alternative way of thinking about this problem we should step back and think about what we want from the solution. Fundamentally, a continuation is a function from a value input or an exceptional input, to a value output or an exceptional output.

```
set_value -----\                              /----- return_value
               |----- Perform Operation -----|
set_exception -/                              \----- return_exception
```

This basic structure covers all of the above uses. The question becomes how we can build this in an efficient manner?

One option is to do what we do in a lot of Facebook's code, and implement the operation in terms of `folly::Try<T>` putting all functionality in the continuation itself. Unfortunately, it is clumsy to write efficient code that wants to ignore one or other path entirely using this structure. We are forced into the combined structure in the code.

Abstractly, though, if we assume that these operations are inputs and outputs from some class, we see that the input is a `Promise` type:

```
                                    /----- return_value
Promise ----- Perform Operation -----|
                                    \----- return_exception
```

Where a promise is a class concept consisting of two `void`-returning functions: `set_value` and `set_exception`.

Taking a further look at this we realise that actually the output path is merely the input to another operation - one owned by the executor itself. So we see another `Promise` lying in wait for us:

```
Promise ----- Perform Operation ----- Promise
```

Fundamentally, then, each of the continuation constructors should produce something that has a promise as input, and a promise as output, and where the value and error operations can be mixed based on the implementation. This is fully general. Moreover, by requiring that both of these functions be provided and thus called by the

implementation, it is also safe because the compiler will fail if a function fails to compile. The `set_value` input can map to either the `return_value` or `return_exception` output, and similarly for the `set_exception` input.

So what does this look like? Because the Promises are both concepts, not types, we need to be able to generate this code. The input promise is a feature of the task we construct. This much is simple. In addition, we need a way to take the output promise as an input. That is, we need to construct a usable task from some partial task, plus a promise.

In summary:

> The continuation is an object that, when passed a Promise as a parameter constructs a new object that is itself a Promise.

§ 5. Examples

Let's take a few examples of what this looks like to implement.

Given a continuation provided by some continuation construction function (some examples of which we see below) and passed to our processing function, we can use the continuation by: 1) constructing internal `Promise` that is tied to our output `Future` 2) pass the output promise to the continuation as a means of constructing a viable promise object. 3) call the appropriate operation on the input with data from our input `Future`.

In code that general principle looks like:

```
OutputFuture process_continuation(FutureContinuation&& continuation) {
  // Construct output promise/future contract
  [outputPromise, outputFuture] make_promise_contract<T>();

  // Construct the input promise by parameterising the continuation with the
  // output promise.
  auto inputPromise = std::move(continuation)(outputPromise);

  // Call the appropriate input data path on the input promise
  if(have_value()) {
    std::move(inputPromise).set_value(value());
  } else {
    std::move(inputPromise).set_exception(exception());
  }

  // Return the outputFuture that will include the result of the computation
  return outputFuture;
}
```

§ 5.1. on_value

`on_value` takes a function from a value to a value and, as discussed above, returns a function that can be passed a Promise, and which constructs a Promise:

```
// F = function(int(int))
template <typename F>
auto on_value(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
          auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise)) {}

            void set_value(int value) {
                try {
                    auto resultOfOperation = f_(value);
                    outputPromise_.set_value(resultOfOperation);
                } catch (...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }

            void set_exception(std::exception_ptr e) {
                outputPromise_.set_exception(std::move(e));
            }

        private:
            F f_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(continuationFunction),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}
```

and constructs the continuation as:

```
auto continuation = on_value([](int x) {return x*2;});
```

As an example of using these constructs as if as a simple callback, for an executor that only supports simple callback mechanisms, we can see that all of this code optimises to nothing (https://godbolt.org/g/m3qvoj).

§ 5.2. on_error

Here we construct a continuation from a function from exception_ptr to exception_ptr as a means of only processing our error stream.

```
// F = function(exception_ptr(exception_ptr))
template <typename F>
auto on_error(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
          auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise)) {}

            void set_value(int value) {
                outputPromise_.set_value(value);
            }

            void set_exception(std::exception_ptr e) {
                try {
                    auto resultOfOperation = f_(std::move(e));
                    // Set the exception from the return value
                    outputPromise_.set_exception(resultOfOperation);
                } catch (...) {
                    // Also catch the error for completeness.
                    outputPromise_.set_exception(std::current_exception());
                }
            }

        private:
            F f_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(continuationFunction),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}
```

We can construct a simple exception processing continuation as:

```
auto continuation = on_error([](std::exception_ptr e) {
  std::cerr << "Log!\n"; return e;});
```

Note that even here, if we do not end up using the exception path, all of this optimises away (https://godbolt.org/g/xRm2oH)

§ 5.3. on_variant

We can implement a version that passes variants through as:

```
// F = function(variant<int, exception_ptr>(variant<int, exception_ptr>))
template <typename F>
auto on_variant(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
          auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise)) {}

            void set_value(int value) {
                apply(value);
            }

            void set_exception(std::exception_ptr e) {
                apply(std::move(e));
            }

        private:
            F f_;
            OutputPromise outputPromise_;

            void apply(std::variant<int, std::exception_ptr> v) {
                struct visitor {
                    void operator()(int result) {
                        outputPromise_.set_value(std::move(result));
                    }
                    void operator()(std::exception_ptr ex) {
                        outputPromise_.set_exception(std::move(ex));
                    }
                    OutputPromise& outputPromise_;
                };
                try {
                    auto intermediateValue = f_(std::move(v));
                    std::visit(visitor{outputPromise_}, std::move(intermediateValue)
                } catch(...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }
        };

        return InputPromise(
            std::move(continuationFunction),
            std::forward<OutputPromiseRef>(outputPromise));
```

```
    };
}
```

Constructing the continuation with:

```
struct visitor {
    std::variant<int, std::exception_ptr>
    operator()(int val) const {
      return val + 1;
    }

    std::variant<int, std::exception_ptr>
    operator()(std::exception_ptr ex) const {
      return ex;
    }
};
auto continuation = on_variant(
    [](std::variant<int, std::exception_ptr> v) -> std::variant<int, std::exception_
        return std::visit(visitor{}, std::move(v));
    });
```

Again, with use of variants, if we do not actually use the `exception_ptr` route this optimises away (https://godbolt.org/g/AZRAeK).

§ 5.4. on_value_logging_error

Finally, we can build an operation that takes two functions, where the error handler simply passes through the exception with logging:

```
// F = function(int(int))
template <typename F, typename FE>
auto on_value_log_exception(F&& valueContinuation, FE&& errorContinuation) {
    return [valueContinuation = std::forward<F>(valueContinuation),
            errorContinuation = std::forward<FE>(errorContinuation)](
          auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type

        class InputPromise {
        public:
            InputPromise(
                F&& f, FE&& fe, OutputPromise&& outputPromise) :
                f_(std::move(f)), fe_(std::move(fe)), outputPromise_(std::move(outpu

            void set_value(int value) {
                try {
                    auto resultOfOperation = f_(value);
                    outputPromise_.set_value(resultOfOperation);
                } catch (...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }

            void set_exception(std::exception_ptr e) {
                outputPromise_.set_exception(fe_(std::move(e)));
            }

        private:
            F f_;
            FE fe_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(valueContinuation),
            std::move(errorContinuation),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}
```

and where we might construct this as:

```
auto continuation = on_value_log_exception(
    [](int x) {return x*2;},
    [](std::exception_ptr e){std::cerr << "Have exception\n"; return e;});
```

Note that with improvements to `exception_ptr` this is where we could benefit from snooping on the exception without rethrow, as `folly::exception_wrapper` enables or is proposed in p1066. Full source example: https://godbolt.org/g/Xbp5xK.

§ 6. Noexcept

The methods on `FutureContinuation` should be noexcept. Any exception handling should be handled as part of the `FutureContinuation` task and passed to the `set_exception` output.

With this change, the executors do not need exception propagation properties, nor do they need to expose queries that specify what happens when an exception leaks from a continuation because this cannot happen. This is a considerable simplification and reduction in committee work that is still in progress.

§ 7. Rethinking bulk execution

We should encode bulk operations as extended continuations. Bulk execution is a property of a task, not an executor. While we realise that the executor has influence on how the task runs and where, which may include how it is compiled to do bulk dispatch, the actual properties of the task are orthogonal to the API exposed by executors. To put it another way, the flow of data through the graph is orthogonal to whether a given node in that graph offers fork-join or other forms of execution.

Encoding the bulk functionality as part of the continuation, rather than the executor API, would allow us to halve the number of executor entry points. Further, bulk continuations need not be part of the fundamental concepts and instead we can encode the interface simply as an extended set of task construction functions as in p1054. It also means we do not have to complicated the definition of Futures to add bulk APIs; the task passed to a future can be transparently passed to the underlying executor and the future does not care whether it is a bulk task or not.

§ 7.1. Bulk as in P0443

Abstracted as a task and used in the executor or future API, a bulk operation would look like the following:

```
auto continuation = bulk_on_value(
    [](
        const int& input,
        int /*idx*/,
        atomic<int>& /*shared*/,
        int &out){
      out+=input;
    }, // Task
    int{20}, // Shape
    []() -> atomic<int> {return {0};}, // Shared factory
    []() -> int {return 0;} // result factory);

{
  // As a task passed to an executor
  TrivialFuture<int> inputFuture{2};
  auto executorResultF =
    BulkExecutor{}.then_execute(continuation, std::move(inputFuture));
  std::cout << std::this_thread::future_get(std::move(executorResultF)) << "\n";
}
{
  // As a task passed to a future
  TrivialFuture<int> inputFuture{2};
  auto futureResultF = std::move(inputFuture).then(continuation);
  std::cout << std::this_thread::future_get(std::move(futureResultF)) << "\n";
}
```

The mechanism here is an extension of the non-bulk continuation type. Where for non-bulk tasks the continuation was:

> An object that, when passed a Promise as a parameter constructs a new object that is itself a Promise.

Bulk is similar, but a little more complicated to allow an arbitrary executor to hook up the bulk algorithm.

> An object that, when passed a Promise as a parameter constructs a new object that, when passed a BulkDriver as a parameter, returns a promise.

That is that we add a stage to the setup algorithm that allows the executor to configure the execution.

`bulk_on_value` becomes a function that returns a nested type:

```
template<class F, class Shape, class RF>
auto bulk_on_value(
    F&& continuationFunction,
    Shape s,
    RF&& resultFactory) {
  return [continuationFunction = std::forward<F>(continuationFunction),
          resultFactory = std::forward<RF>(resultFactory),
          s = std::forward<Shape>(s)](
        auto&& outputPromise) mutable {
    using OutputPromiseRef = decltype(outputPromise);
    using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type;

    return [continuationFunction = std::forward<F>(continuationFunction),
            resultFactory = std::forward<RF>(resultFactory),
            s = std::forward<Shape>(s),
            outputPromise = std::forward<OutputPromiseRef>(outputPromise)](
          auto&& bulkDriver) mutable {
      using BulkDriverRef = decltype(bulkDriver);
      using BulkDriver = typename std::remove_reference<BulkDriverRef>::type;

      return InputPromise<F, OutputPromise, Shape, RF, BulkDriver>(
          std::move(continuationFunction),
          std::forward<OutputPromiseRef>(outputPromise),
          resultFactory(),
          s,
          std::forward<decltype(bulkDriver)>(bulkDriver));
    };
  };
}
```

all this does is pass through the data, but defer binding of the output promise type and bulk driver type, effectively to construction of a later object. This approach ensures the greatest level of type visibility to the compiler and the most efficient, allocation-free code.

The bulk driver exposes an interface that a given task constructor can hook in to, but is under control of the executor such that the implementation of the operations is under the control of the target. For example, a very simple driver that runs the entire algorithm when the executor calls the `end()` method on the driver might look like:

```
template<class PromiseT, class ShapeF, class AtF, class DoneF>
struct EndDriverImpl {
  void start() {
  }

  void end() {
    const auto& shape = shapeF_();
    for(std::decay_t<decltype(shape)> i = 0; i < shape; ++i) {
      atF_(i);
    }
    doneF_();
  }

  PromiseT& promise_;
  ShapeF shapeF_;
  AtF atF_;
  DoneF doneF_;
};

struct EndDriver {
  template<class PromiseT, class ShapeF, class AtF, class DoneF>
  auto operator()(PromiseT& prom, ShapeF&& shapeF, AtF&& atF, DoneF&& doneF){
    return EndDriverImpl<PromiseT, ShapeF, AtF, DoneF>{
      prom,
      std::forward<ShapeF>(shapeF),
      std::forward<AtF>(atF),
      std::forward<DoneF>(doneF)};
  }
};
```

We add to the promise interface a `get_driver()` method that will return the driver:

```
auto bulk_driver() {
  return bulkDriver_(
    *this,
    [this](){return this->get_shape();},
    [this](auto i){this->execute_at(i);},
    [this](){this->done();});
}
```

Remember, the Promise was constructed by the task - so the means of constructing the driver is under task control, but the driver type is under Executor control. The executor then is free to setup and run the algorithm as it sees fit, calling the functions passed to the driver's constructor through whatever means necessary. For example, when the input future is satisfied and the executor knows the value is ready, it might call:

```
auto driver = boundCont.bulk_driver();
boundCont.set_value(std::move(inputFuture).get());
driver.start();
driver.end();
```

such that the call to end() runs the above sequence of calls into the task. That `end()` method could of course be implemented as an openmp parallel loop, a CUDA dispatch or any similar operation.

The BulkDriver is defined as being a function that takes:

- A reference to the input promise (a reference to the current continuation).
- A nullary function that when called, returns the shape.
- A function that executes the continuation at some point in the shape, and takes an element from the shape range.
- A function to call on completion of the bulk execution.

§ 7.2. An opportunity: rethinking the bulk parameterisation

Abstracting the execution algorithm offers us the opportunity to be flexible about the design of the continuation.

As one example, when we were considering the design of bulk, we realised that there are some limitations:

- The shape is defined only at graph construction time.
- Shared state and output can not be constructed dependent on the input.
- The output has to map directly to the output future. This makes it hard to, for example, perform a final accumulation into an atomic because atomics are not movable.

We therefore propose extending the definition of the continuation:

- Instead of a static shape we have a shape factory, that also takes the input by reference. Note that this can be constexpr and ignore its input to regain the original behaviour.
- The shared state factory be extended to take the shape and input value.
- The output factory be replaced by an output selector, parameterised by the shared state and output promise, such that the output is transferred from the shared state to the output promise.

Most importantly, this proposal is merely for the bulk_on_value continuation construction function we might consider putting into the standard library. It does not require a change to the BulkDriver design - the design is multi stage which allows the continuation to interact however it needs to.

Instead we might modify `bulk_on_value` to be constructed as follows:

```
auto continuation = bulk_on_value(
    [](const InputT& input, ShapeElementT /*idx*/, SharedStateT &shared){*shared+=in
    [](const InputT& /*input value*/){return int{20};}, // Shape factory.
    [](const ShapeT& /*shape*/, const InputT& /*input value*/) -> SharedStateT {retu
    [](SharedStateT&& shared, auto& outputPromise) {  // Result selector/output
      outputPromise.set_value(std::move(*shared));
    });
```

This only changes the implementation of the task - the basic algorithm the `BulkDriver` executes need not change if we define it correctly.

§ 8. Proposed New Wording for P0443

§ 8.1. `Promise` requirements

A type `P` meets the `Promise` requirements for some value type `T` if an instance `p` of `P` satisfies the requirements in the table below.

Expression	Return Type	Operational semantics
`p.set_value(T&&)`	void	Defined if T is not void. Completes the promise with a value. Should accept by forwarding reference.
`p.set_value()`	void	Defined if T is void. Completes the promise with no value.
`p.set_exception(std::exception_ptr)`	void	Completes the promise with an exception wrapped in a `std::exception_ptr`.

§ 8.2. OneWayFutureContinuation requirements

A type `OFC` meets the OneWayFutureContinuation requirements if `OFC` satisfies the requirements of `MoveConstructible` and for an instance `ofc` of `OFC`, `INVOKE(std::forward<OFC>(ofc))` is valid.

§ 8.3. TwoWayFutureContinuation requirements

A type `TFC` meets the `TwoWayFutureContinuation` requirements if `TFC` satisfies the requirements of `MoveConstructible` and for an instance `tfc` of `TFC` and a value `p` whose type, `P` satisfies the `Promise` requirements and for which `INVOKE(std::forward<P>(p))` is valid.

§ 8.4. ThenFutureContinuation requirements

A type `THFC` meets the `TwoWayFutureContinuation` requirements if `THFC` satisfies the requirements of `MoveConstructible`, and for an instance `thfc` of `TFC` and a value `p` whose type, `P` that satisfies the `Promise` requirements and where `INVOKE(std::forward<P>(p))` is valid and returns a value `pout` of type `POUT` that satisfies the `Promise` requirements for some value type `TIn`, potentially `void`, that is known to the caller.

§ 8.5. Changes to OneWayExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X` and `f` denotes an object of type `F&&` that satisfies the requirements of `OneWayFutureContinuation`.

§ 8.6. Changes to TwoWayExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `f` denotes a an object of type `F&&` that satisfies the requirements of `TwoWayFutureContinuation` and `R` is `void` or denotes the value type of a value `p` of type `P&&` that satisfies the `Promise` requirements for value type `R` and that may be passed to the expression `DECAY_COPY(std::forward<F>(f))(std::move(p))`.

Expression	Return Type	Operational semantics
`p.twoway_execute(f)`	A type that satisfies the Future requirements for the value type `R`.	Creates an execution agent which invokes `DECAY_COPY( std::forward<F>(f))(p)` for some value `p` of type `P` that satisfies the requirements of `Promise` for value type `R`, with the call to `DECAY_COPY` being evaluated in the thread that called `twoway_execute`. May block pending completion of DECAY_COPY(std::forward (f))(p). The invocation of twoway_execute synchronizes with (C++Std [intro.multithread]) the invocation of f. Stores the result of a call to `p.set_value(r)`, `p.set_value()`, or `p.set_exception(e)` for `r` of type `R` or `e` of type `std::exception_ptr` in the associated shared state of the resulting `Future`.

§ 8.7. Changes to ThenExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `fut` denotes a future object satisfying the `Future` requirements, `f` denotes a function object of type `F&&` that satisfies the requirements of `ThenFutureContinuation` and `R` is `void` or denotes the value type of a value `p` of type `P&&` that satisfies the

`Promise` requirements for value type `R` and that may be passed to the expression `DECAY_COPY(std::forward<F>(f))(std::move(p))`.

Expression	Return Type	Operational semantics
`p.then_execute(f, fut)`	A type that satisfies the Future requirements for the value type `R`.	When `fut` is ready, creates an execution agent which invokes `DECAY_COPY( std::forward<F>(f))(p)` for some value `p` of type `P` that satisfies the `Promise` requirements for value type `R`, with the call to `DECAY_COPY` being evaluated in the thread that called then_execute. May block pending completion of `DECAY_COPY( std::forward<F>(f))(p)`. The invocation of `then_execute` synchronizes with (C++Std [intro.multithread]) the invocation of `f`. If `fut` is ready with a value, calls `f.set_value(r)`, if `fut` is ready and the value is void calls `f.set_value()`. If `fut` is ready with an exception calls `f.set_exception(e)`. Stores the result of a call to `p.set_value(r)`, `p.set_value()`, or `p.set_exception(e)` for `r` of type `R` or `e` of type `std::exception_ptr` in the associated shared state of the resulting `Future`.

§ 8.8. Changes to Bulk requirements in general

Precise wording TBD. Roughly:

- Remove bulk APIs from executor concepts.
- Define bulk continuation concepts as generalisations of the non-bulk continuation concepts.
- Define bulk driver concept.

This will cleanly extend to adding a wider set of abstractions beyond basic bulk, orthogonally to the basic executor interface.